

## **Chapter 3**

# **Overview of 8086 Microprocessor**

Credit: 3 hrs

# Overview:

1. Architecture of 8086 Microprocessor
2. Addressing modes
3. ALP Development Tools: Editor, Assembler and Linker

# 1. Introduction to 8086 Microprocessor

- 8086 Microprocessor is an **enhanced version of 8085** Microprocessor that was designed by Intel in 1976.
- It is a **16-bit Microprocessor** having:
  - **20 address lines** and
  - **16 data lines** that provides up to 1MB storage.
- It consists of powerful instruction set, which provides operations like multiplication and division easily.
- It **supports two modes of operation**, i.e. Maximum mode and Minimum mode.
  - **Maximum mode** is suitable for system having **multiple processors** and
  - **Minimum mode** is suitable for system having a **single processor**.

# Features of 8086 MP

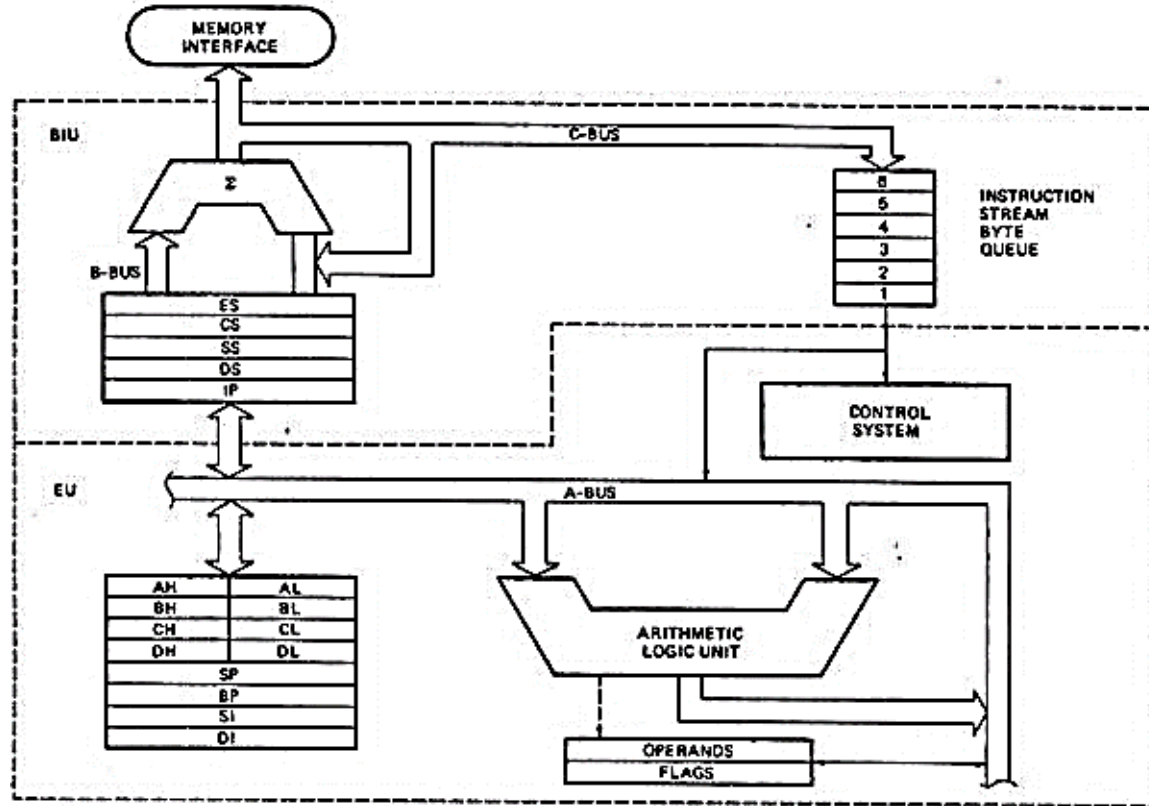
1. It was the **first 16-bit microprocessor**.
2. It has a 16-bit data bus, so it can read data from or write data to memory and ports either 16-bit or 8-bit at a time.
3. It has **20 bit address bus** and can access up to 2<sup>20</sup> memory locations (1 MB).
4. It can **support up to 64K I/O ports**.
5. It provides **fourteen 16-bit registers**.
6. It has **multiplexed address and data bus** AD0-AD15 & A16-A19
7. It **requires single phase clock** with 33% duty cycle to provide internal timing.
8. Prefetches up to 6 instruction bytes from memory and queues them in order to speed up the processing.
9. 8086 **supports two modes of operation**: Minimum mode and Maximum mode

# Comparison between 8085 & 8086 Microprocessor

- **Size** – 8085 is 8-bit microprocessor, whereas 8086 is 16-bit microprocessor.
- **Address Bus** – 8085 has 16-bit address bus while 8086 has 20-bit address bus.
- **Memory** – 8085 can access up to 64Kb, whereas 8086 can access up to 1 Mb of memory.
- **Instruction** – 8085 doesn't have an instruction queue, whereas 8086 has an instruction queue.
- **Pipelining** – 8085 doesn't support a pipelined architecture while 8086 supports a pipelined architecture.
- **I/O** – 8085 can address  $2^8 = 256$  I/O's, whereas 8086 can access  $2^{16} = 65,536$  I/O's.
- **Cost** – The cost of 8085 is low whereas that of 8086 is high.

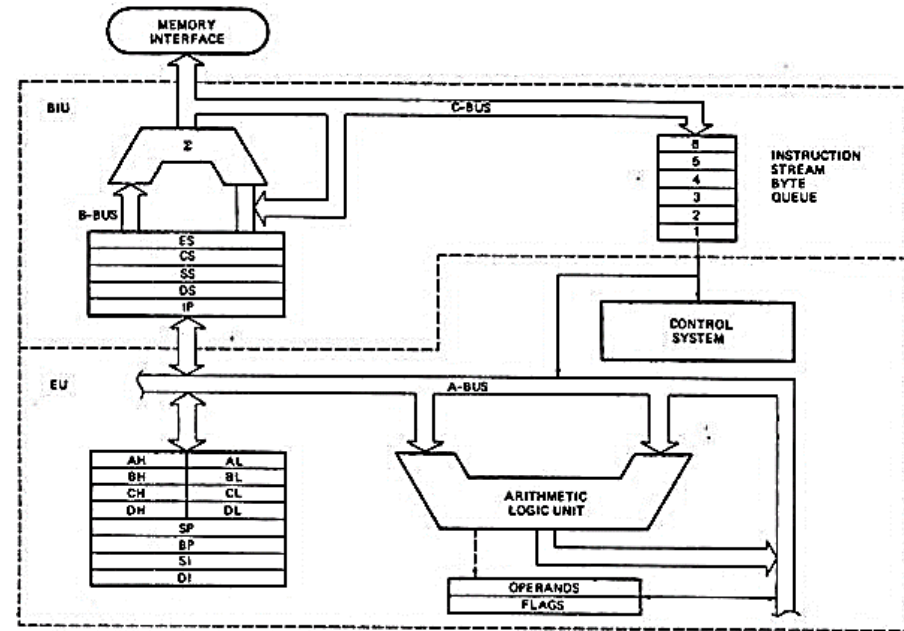
## 2. Architecture of 8086

- As shown in the figure, the 8086 CPU is divided into two independent functional parts:
  - Bus Interface Unit(BIU)
  - Execution Unit(EU)
- Dividing the work between these two units' speeds up processing



### a) The execution unit (EU):

- The execution unit of the 8086 tells the BIU where to fetch instructions or data from, decodes instructions, and executes instructions.
- The EU contains control circuitry, which directs internal operations.
- A decoder in the EU translates instructions fetched from memory into a series of actions, which the EU carries out.
- The EU has a 16-bit arithmetic logic unit (ALU) which can add, subtract, AND, OR, XOR, increment, decrement, complement or shift binary numbers.
- The main functions of EU are:
  - Decoding of instructions
  - Execution of instructions
    - EU extracts instructions from top of queue in BIU
    - Decode the instructions
    - Generates operands if necessary
    - Passes operands to BIU & requests it to perform read or write bus cycles to memory or I/O
    - Perform the operation specified by the instruction on operands



### b) Bus Interface Unit (BIU):

- The BIU:
  - sends out addresses,
  - fetches instructions from memory,
  - reads data from ports and memory, and
  - writes data to ports and memory.
- In simple words, the BIU handles all transfers of data and addresses on the buses for the execution unit.

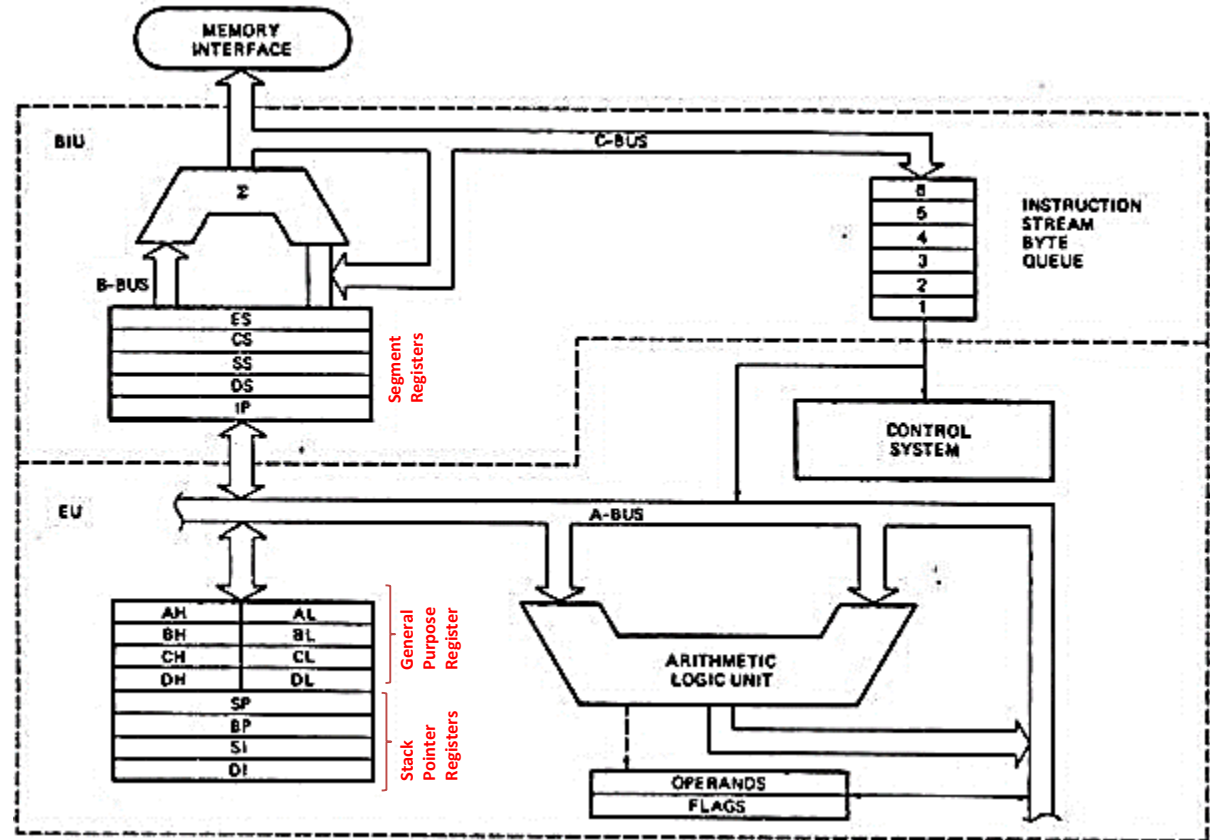
# Functional parts of 8086 MP

## 1. Execution Unit (EU)

- a) ALU
- b) General Purpose Registers
- c) Stack Pointer Registers
- d) Flag Registers
- e) Control System

## 2. Bus Interface Unit (BIU)

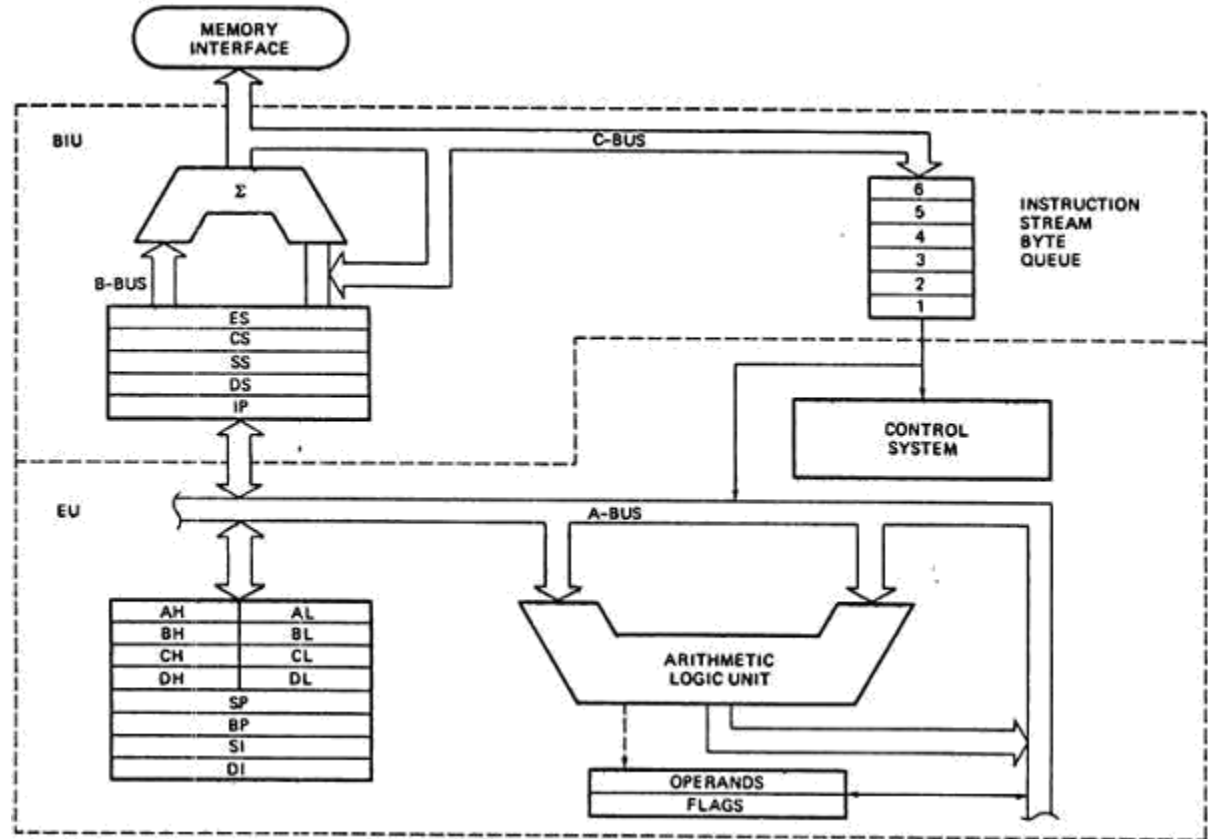
- a) Instruction Queue
- b) Segment Registers
- c) Interface Unit



# 1. Execution Unit

## a) ALU

- It handles all arithmetic and logical operations, like +, -, ×, /, OR, AND, NOT operations.



## b) General purpose register

- There are 8 general purpose registers, i.e., AH, AL, BH, BL, CH, CL, DH, and DL.
- These registers can be used individually to store 8-bit data and can be used in pairs to store 16-bit data.
- The valid register pairs are AH and AL, BH and BL, CH and CL, and DH and DL.
- It is referred to the AX, BX, CX, and DX respectively.
  - **AX register:**
    - It is also known as accumulator register.
    - It is used to store operands for arithmetic operations.
  - **BX register**
    - It is used as a base register.
    - It is used to store the starting base address of the memory area within the data segment.
  - **CX register**
    - It is referred to as counter.
    - It is used in loop instruction to store the loop counter.
  - **DX register**
    - This register is used to hold I/O port address for I/O instruction.

|                                              |   |   |    |   |   |
|----------------------------------------------|---|---|----|---|---|
| 15                                           | H | 8 | 7  | L | 0 |
| AX (Accumulator)                             |   |   |    |   |   |
| AH                                           |   |   | AL |   |   |
| BX (Base Register)                           |   |   |    |   |   |
| BH                                           |   |   | BL |   |   |
| CX (Used as a counter)                       |   |   |    |   |   |
| CH                                           |   |   | CL |   |   |
| DX (Used to point to data in I/O operations) |   |   |    |   |   |
| DH                                           |   |   | DL |   |   |

## c) Pointer and Index Registers

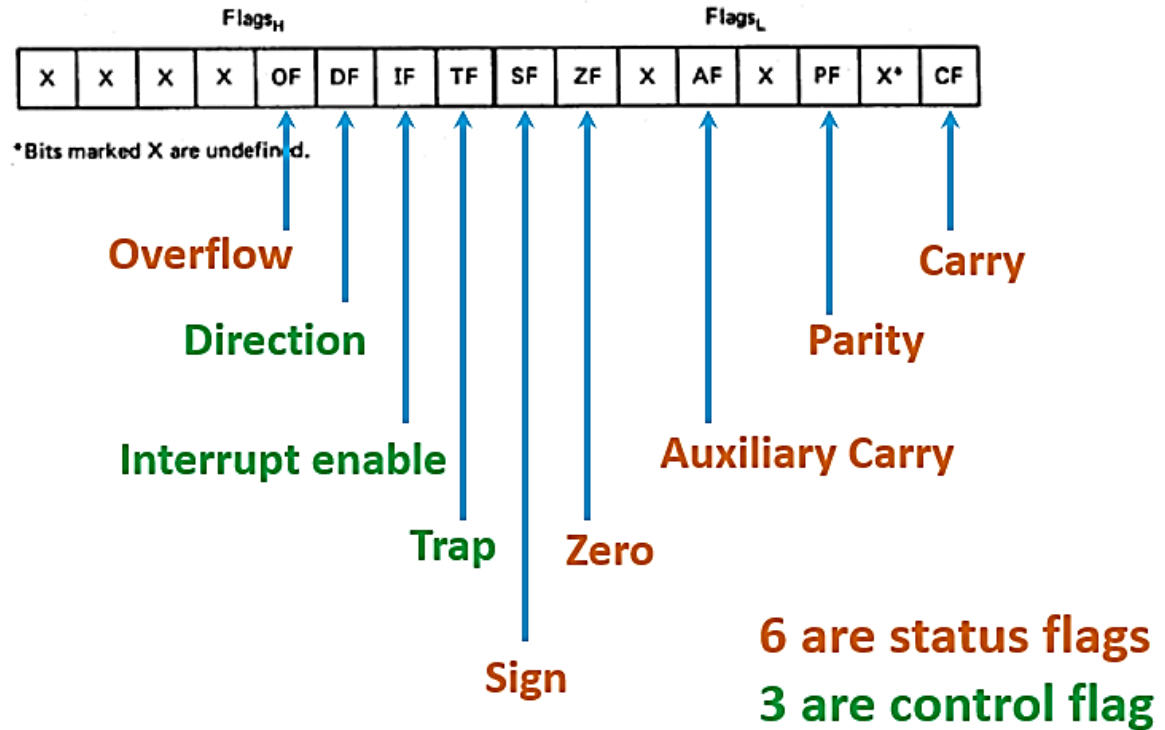
|    |                     |
|----|---------------------|
| SP | Stack Pointer       |
| BP | Base Pointer        |
| SI | Source Index        |
| DI | Destination Index   |
| IP | Instruction Pointer |

- All 16 bits wide, L/H bytes are not accessible
- Used as memory pointers
  - Example: MOV AH, [SI]
    - *Move the byte stored in memory location whose address is contained in register SI to register AH*
- IP is not under direct control of the programmer

- **Stack pointer register**
  - It is a 16-bit register, which **holds the address** from the start of the segment to the memory location, where a word was most recently stored on the stack.
- **Instruction pointer**
  - It is a 16-bit register **used to hold the address of the next instruction** to be executed.

## d) Flag Register

- It is a 16-bit register that **behaves like a flip-flop**, i.e. it changes its status according to the result stored in the accumulator.
- It **has 9 flags** and they are divided into 2 groups –
  - Conditional Flags and
  - Control Flags.



## Conditional Flags

- It represents the result of the last arithmetic or logical instruction executed. Following is the list of conditional flags –
  - **Carry flag**
    - This flag **indicates an overflow condition** for arithmetic operations.
  - **Auxiliary flag:**
    - When an operation is performed at ALU, it results in a carry/borrow from lower nibble (i.e. D0 – D3) to upper nibble (i.e. D4 – D7), **then this flag is set**, i.e. carry given by D3 bit to D4 is AF flag.
    - The processor uses this flag to perform binary to BCD conversion.
  - **Parity flag :**
    - This flag is used to **indicate the parity of the result**, i.e. when the lower order 8-bits of the result contains even number of 1's, then the Parity Flag is set. For odd number of 1's, the Parity Flag is reset.
  - **Zero flag :**
    - This flag is **set to 1 when the result of arithmetic or logical operation is zero** else it is set to 0.
  - **Sign flag :**
    - This flag holds the sign of the result, i.e. **when the result of the operation is negative, then the sign flag is set to 1** else set to 0.
  - **Overflow flag :**
    - This flag represents the result **when the system capacity is exceeded**.

# Control Flags

- Control flags controls the operations of the execution unit. Following is the list of control flags –
  - **Trap flag:**
    - It is **used for single step control** and allows the user to execute one instruction at a time for debugging.
    - If it is set, then the program can be run in a single step mode.
  - **Interrupt flag :**
    - It is **an interrupt enable/disable flag**, i.e. used to allow/prohibit the interruption of a program.
    - It is **set to 1 for interrupt enabled condition** and set to 0 for interrupt disabled condition.
  - **Direction flag :**
    - It is **used in string operation**.
    - As the name suggests when it is set then string bytes are accessed from the higher memory address to the lower memory address and vice-a-versa.

## 2. Bus Interface Unit (BIU)

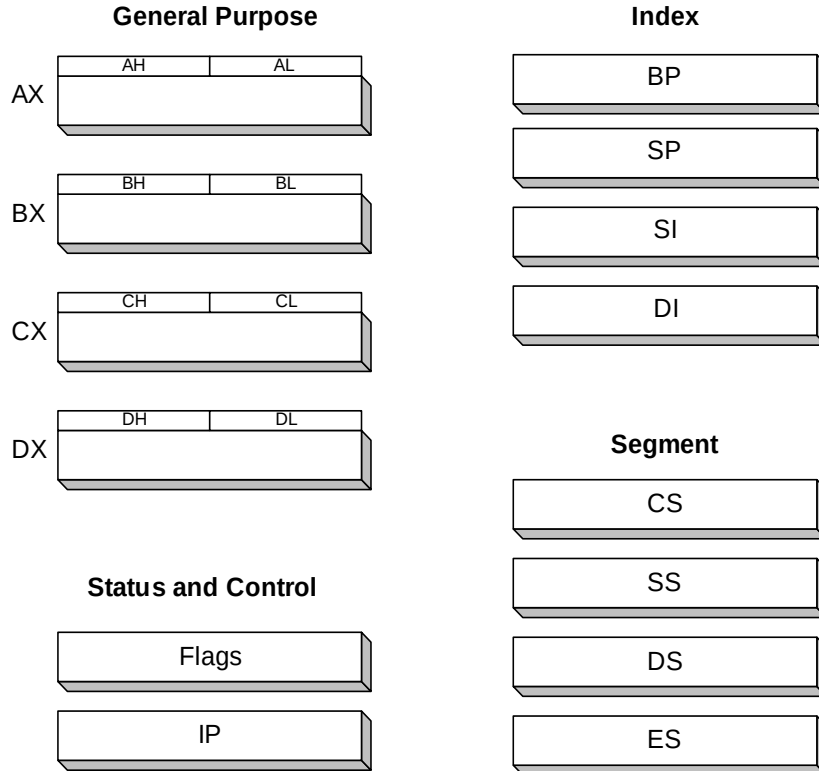
### a) Instruction queue

- BIU contains the instruction queue.
- BIU gets upto 6 bytes of next instructions and stores them in the instruction queue.
- When EU executes instructions and is ready for its next instruction, then it **simply reads the instruction from this instruction queue resulting in increased execution speed.**

## b) Segment Register

- It holds the addresses of instructions and data in memory, which are used by the processor to access memory locations.
- BIU has 4 segment buses, i.e. CS, DS, SS& ES.
  - **CS (Code Segment):** It is used for addressing a memory location in the code segment of the memory, where the executable program is stored.
  - **DS (Data Segment):** It consists of data used by the program and is accessed in the data segment by an offset address or the content of other register that holds the offset address.
  - **SS (Stack Segment):** It handles memory to store data and addresses during execution.
  - **ES (Extra Segment):** ES is additional data segment, which is used by the string to hold the extra destination data.
- It also contains 1 pointer register IP, which holds the address of the next instruction to be executed by the EU.

# 8086 Registers



This has been already discussed ahead.

# 4. Pin Diagram of 8086 MP

## 1. Power supply

- It uses **5V DC supply** at **VCC** pin 40, and
- uses **ground at VSS** pin 1 and 20 for its operation.

## 2. Clock signal

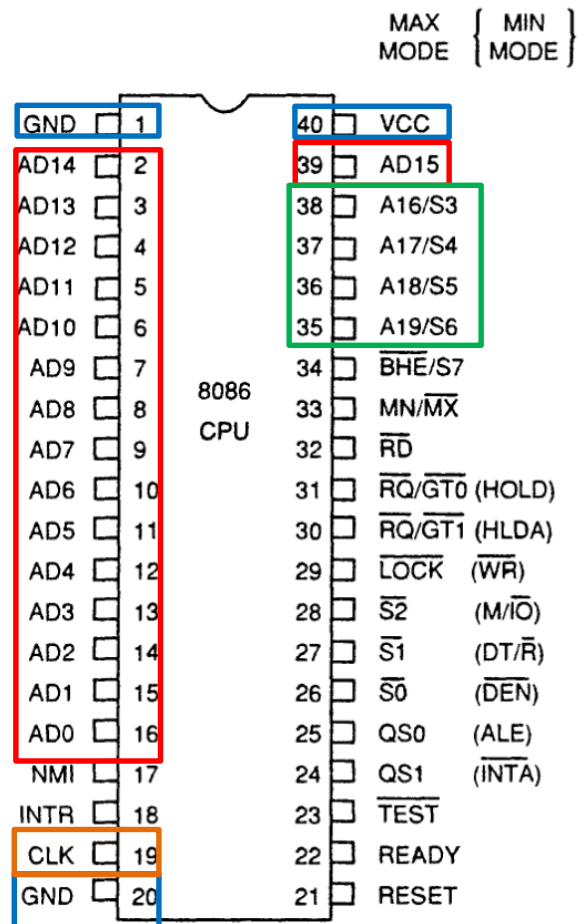
- Clock signal is provided through Pin-19.
- It **provides timing** to the processor for operations.
- Its frequency is different for different versions, i.e. 5MHz, 8MHz and 10MHz.

## 3. Address/data bus

- AD0-AD15.
- These **are 16 address/data bus**.
- AD0-AD7 carries low order byte data and
- AD8-AD15 carries higher order byte data.
- During the first clock cycle, it carries 16-bit address and after that it carries 16-bit data.

## 4. Address/status bus

- A16-A19/S3-S6.
- These are the **4 address/status buses**.
- During the first clock cycle, it carries 4-bit address and later it carries status signals.



## 5. S7/BHE

- BHE stands for Bus High Enable.
- It is available at pin 34 and **used to indicate the transfer of data** using data bus D8-D15.
- This signal is low during the first clock cycle, thereafter it is active.

## 6. Read

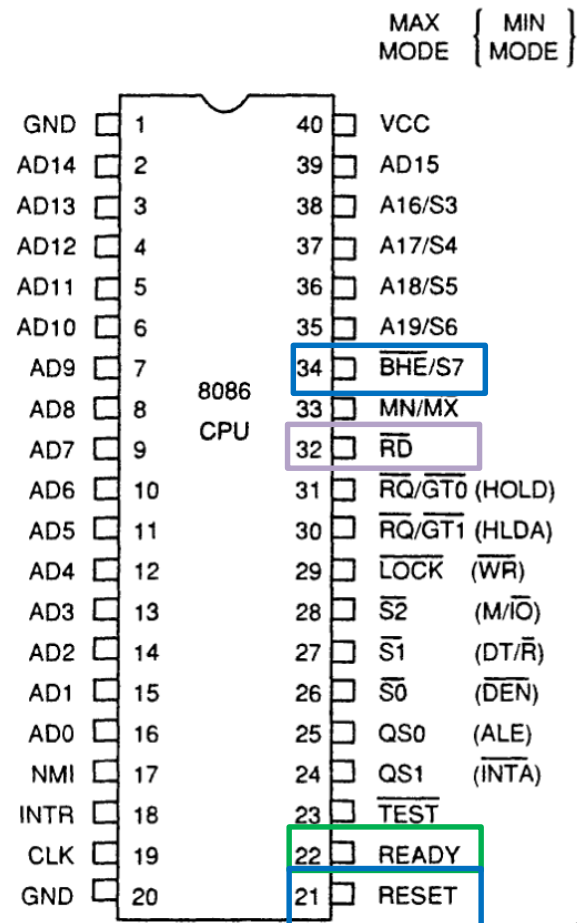
- It is available at pin 32 and is used to read **signal for Read operation**.

## 7. Ready

- It is available at pin 22.
- It is an **acknowledgement signal from I/O devices that data is transferred**.
- It is an active high signal. When it is high, it indicates that the device is ready to transfer data.
- When it is low, it indicates wait state.

## 8. RESET

- It is available at pin 21 and is **used to restart the execution**.
- It causes the processor to immediately terminate its present activity.
- This signal is active high for the first 4 clock cycles to RESET the microprocessor.



## 9. INTR

- It is available at pin 18.
- It is an **interrupt request signal**, which is sampled during the last clock cycle of each instruction to determine if the processor considered this as an interrupt or not.

## 10. NMI

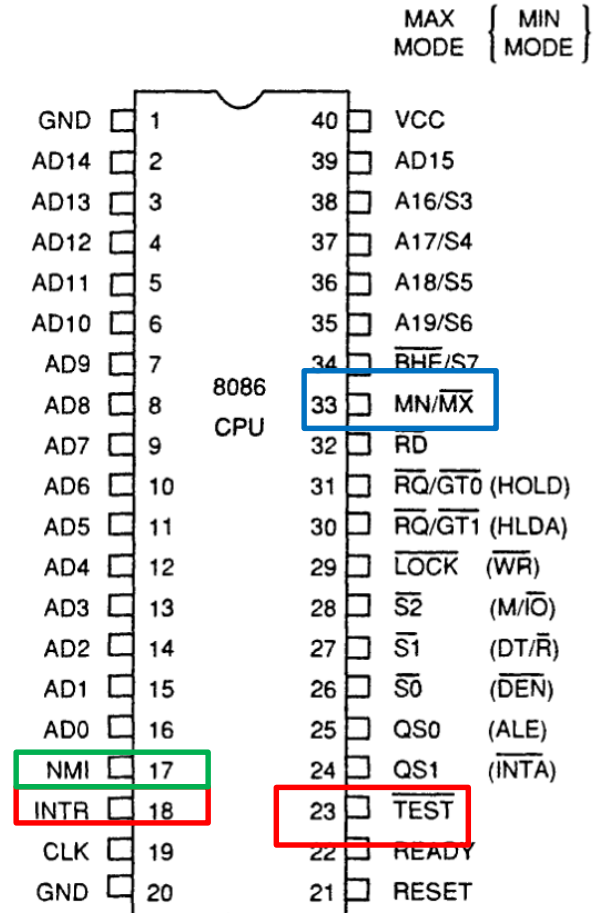
- It stands for **non-maskable interrupt** and is available at pin 17.
- It is an edge triggered input, which **causes an interrupt request to the microprocessor**.

## 11. $\overline{\text{TEST}}$

- This signal is like wait state and is available at pin 23.
- **When this signal is high, then the processor has to wait** for IDLE state, else the execution continues.

## 12. $\text{MN}/\overline{\text{MX}}$

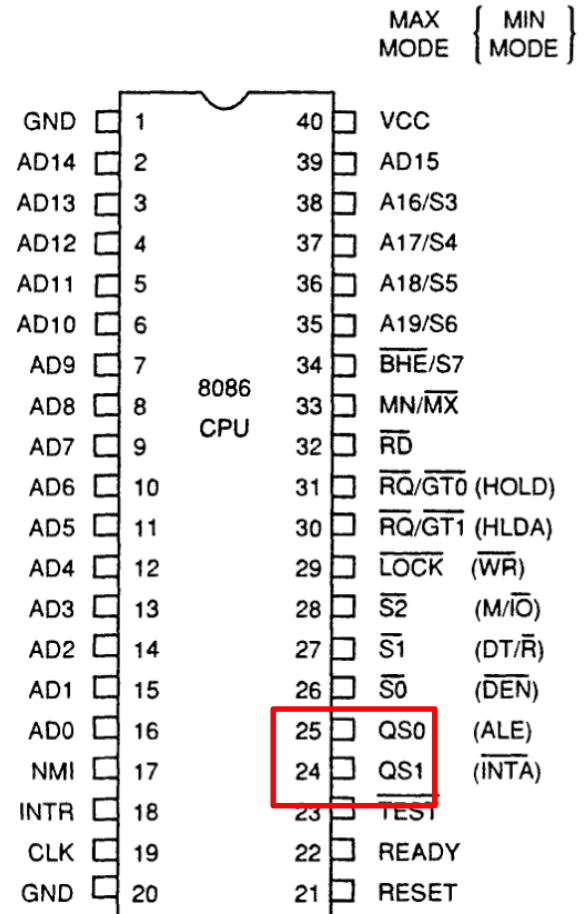
- It stands for Minimum/Maximum and is available at pin 33.
- It **indicates what mode the processor** is to operate in; when it is high, it works in the minimum mode and vice-versa.



### 13. QS1 and QS0

- These are queue status signals and are available at pin 24 and 25.
- These signals **provide the status of instruction queue**

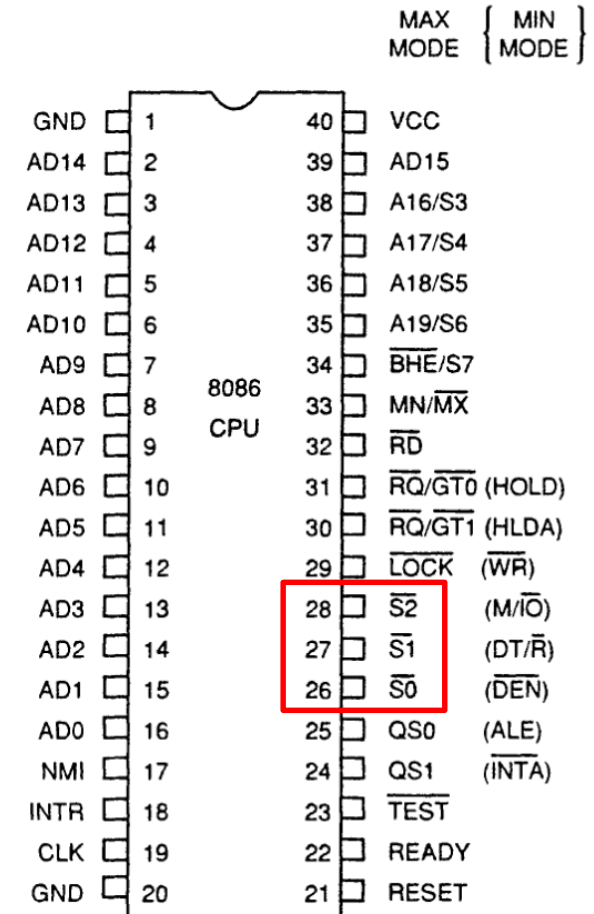
| QS <sub>0</sub> | QS <sub>1</sub> | Status                              |
|-----------------|-----------------|-------------------------------------|
| 0               | 0               | No operation                        |
| 0               | 1               | First byte of opcode from the queue |
| 1               | 0               | Empty the queue                     |
| 1               | 1               | Subsequent byte from the queue      |



## 14. S0, S1, S2

- These are the status signals that **provide the status of operation**, which is used by the Bus Controller 8288 to generate memory & I/O control signals.
- These are available at pin 26, 27, and 28.
- Following is the table showing their status –

| S <sub>2</sub> | S <sub>1</sub> | S <sub>0</sub> | Status                    |
|----------------|----------------|----------------|---------------------------|
| 0              | 0              | 0              | Interrupt acknowledgement |
| 0              | 0              | 1              | I/O Read                  |
| 0              | 1              | 0              | I/O Write                 |
| 0              | 1              | 1              | Halt                      |
| 1              | 0              | 0              | Opcode fetch              |
| 1              | 0              | 1              | Memory read               |
| 1              | 1              | 0              | Memory write              |
| 1              | 1              | 1              | Passive                   |

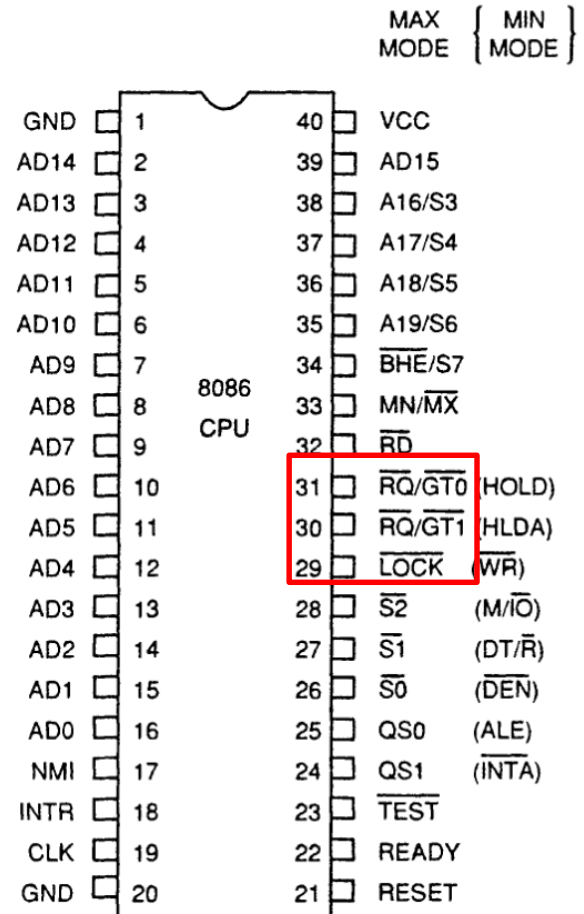


## 15. LOCK

- When this signal is active, it **indicates to the other processors not to ask the CPU to leave the system bus.**
- It is activated using the LOCK prefix on any instruction and is available at pin 29.

## 16. RQ/GT1 and RQ/GT0

- These are the Request/Grant **signals used by the other processors requesting the CPU to release the system bus.**
- When the signal is received by CPU, then it sends acknowledgment.
- RQ/GT0 has a higher priority than RQ/GT1.



## 17. INTA

- It is an **interrupt acknowledgement signal** and is available at pin 24.
- When the microprocessor receives this signal, it acknowledges the interrupt.

## 18. ALE

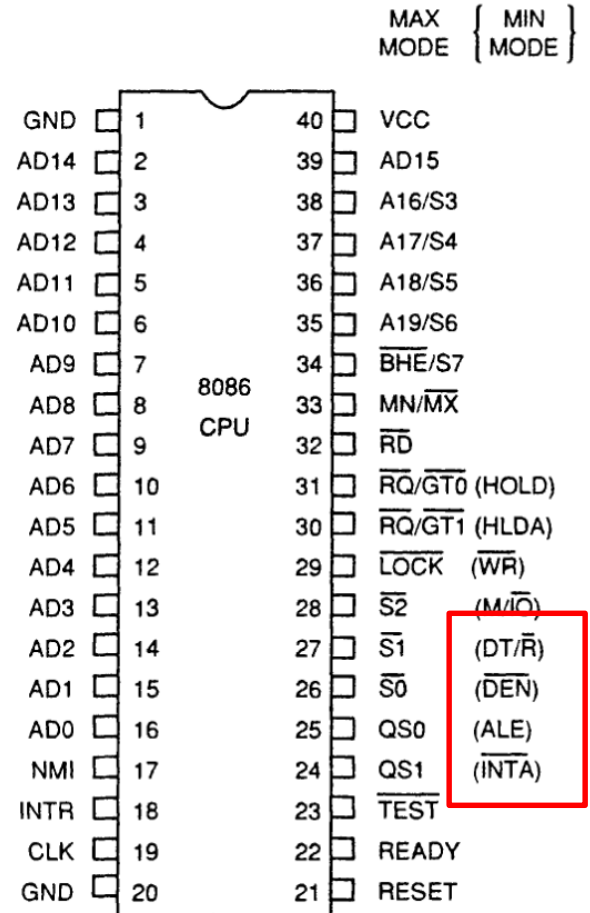
- It stands for address enable latch and is available at pin 25.
- A positive pulse is generated each time the processor begins any operation.
- This signal **indicates the availability of a valid address on the address/data lines.**

## 19. DEN

- It stands for Data Enable and is available at pin 26.
- It is **used to enable Transreceiver 8286.**
- The transceiver is a device used to separate data from the address/data bus.

## 20. DT/R

- It stands for Data Transmit/Receive signal and is available at pin 27.
- It **decides the direction of data flow through the transceiver.**
- When it is **high, data is transmitted out** and vice-a-versa.



## 21. $\overline{M}/\overline{IO}$

- This signal is used to **distinguish between memory and I/O operations.**
- When it is high, it indicates I/O operation and when it is low indicates the memory operation.
- It is available at pin 28.

## 22. $\overline{WR}$

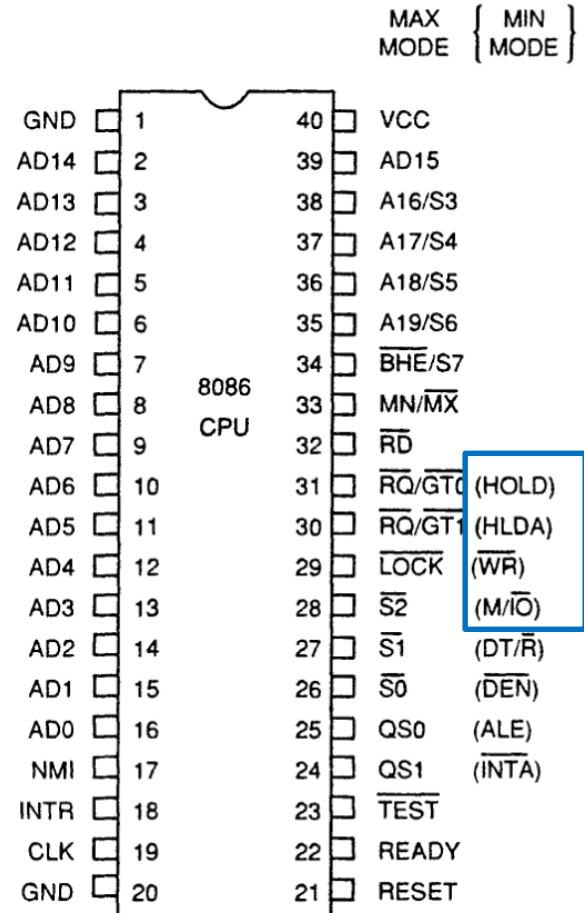
- It **stands for write signal** and is available at pin 29.
- It is used to write the data into the memory or the output device depending on the status of  $\overline{M}/\overline{IO}$  signal.

## 23. $\overline{HLDA}$

- It stands for **Hold Acknowledgement signal** and is available at pin 30.
- This signal acknowledges the HOLD signal.

## 24. $\overline{HOLD}$

- This signal **indicates to the processor that external devices are requesting to access the address/data buses.**
- It is available at pin 31.



# 5. Addressing Modes of 8086

- The different ways in which a source operand is denoted in an instruction is known as **addressing modes**.
- There are 8 different addressing modes in 8086 programming:
  1. Immediate addressing mode
  2. Register addressing mode
  3. Direct addressing mode
  4. Register indirect addressing mode
  5. Based addressing mode
  6. Indexed addressing mode
  7. Based-index addressing mode
  8. Based indexed with displacement mode

## 1. Immediate addressing mode

- The addressing mode in which the **data operand is a part of the instruction itself** is known as immediate addressing mode.
- **Example:**
  - MOV CX, 4929 H
  - ADD AX, 2387 H
  - MOV AL, FFH

## 2. Register addressing mode

- It means that **the register is the source** of an operand for an instruction.
- **Example**
  - MOV CX, AX ; copies the contents of the 16-bit AX register into the 16-bit CX register),
  - ADD BX, AX

## 3. Direct addressing mode

- The addressing mode in which **the effective address of the memory location is written directly** in the instruction.
- **Example**
  - MOV AX, [1592H]
  - MOV AL, [0300H]

## 4. Register indirect addressing mode

- This addressing mode **allows data to be addressed at any memory location through an offset address** held in any of the following registers: BP, BX, DI & SI.
- **Example**
  - MOV AX, [BX] ; Suppose the register BX contains 4895H, then the contents 4895H are moved to AX
  - ADD CX, [BX]

## 5. Based addressing mode

- In this addressing mode, the **offset address** of the operand is given by the **sum of contents of the BX/BP registers and 8-bit/16-bit displacement**.
- **Example**
  - `MOV DX, [BX+04],`
  - `ADD CL, [BX+08]`

## 6. Indexed addressing mode

- In this addressing mode, **the operands offset address is found by adding the contents of SI or DI register and 8-bit/16-bit displacements**.
- **Example**
  - `MOV BX, [SI+16]`
  - `ADD AL, [DI+16]`

## 7. Based-index addressing mode

- In this addressing mode, the offset address of the operand is computed by **summing the base register to the contents of an Index register**.
- **Example**
  - `ADD CX, [BX+SI]`
  - `MOV AX, [BX+DI]`

## 8. Based indexed with displacement mode

- In this addressing mode, the operands offset is computed by **adding the base register contents. An Index registers contents and 8 or 16-bit displacement**.
- **Example**
  - `MOV AX, [BX+DI+08]`
  - `ADD CX, [BX+SI+16]`

# 6. Assembly Language Programming (ALP)

- Assembly language programming is a *low-level programming language* in which the instructions are written in the form of **mnemonics**.
- More simply, we can say the language in the form of mnemonics is known as assembly language.
- Basically **mnemonics are the short form of the words** that are used to specify the operation performed by an instruction.
- **For example:** for addition, the mnemonics used in assembly language is ADD, for transferring data from one location to another, the mnemonics used is MOV.
- This programming language allows the creation of human-readable program equivalent to machine code.

## 7. ALP Development Tools

- Basically, before the fabrication of a microprocessor-based system, the designers first need to design and test the software and hardware of the system.
- So the development system requires a set of tools.
  - Editor
  - Assembler
  - Linker

# 1. Editor

- An editor is a program which **allows us to create a file** containing the assembly language statements for the program.
- Examples of some editors are: PC-Write, Wordstar.
- As we type the program the editor stores the ASCII codes for the letters and numbers in successive RAM locations.
- When we have typed in all your program, we then save the file on the hard disk. This file is called **source file** and the **extension is .asm**.
- **Ex: Sample. asm**

## 2. Assembler

- An assembler program is used to **translate the assembly language mnemonics for instructions to corresponding binary codes.**
- When we run the assembler, it reads the source file of our program from the disk where we have saved it after editing.
- The assembler generates two files:
  - The first file is the **Object file with the extension .OBJ.**
    - The object file consists of the binary codes for the instructions and information about the addresses of the instructions.
    - After further processing, the contents of the file will be loaded in to memory and run.
  - The second file is the **assembler list file with the extension .LST.**

### 3. Linker

- A linker is a program **used to join several object files** into one large object file.
- The linker produces a link file which contains the binary codes for all the combined modules.
- The linker also **produces a link map file** which contains the address information about the linked files (**.exe**).

## Exercise:

1. Explain the 8086 MP architecture with its block diagram.
2. Explain the pin configuration of 8086 in brief.
3. Explain different flags used in 8086 MP.
4. Explain different General-purpose registers in 8086 MP.
5. Explain different segment registers in 8086 MP.
6. Explain the addressing modes of 8086 MP.
7. Compare 8085 and 8086 MPs.
8. What is editor, assembler and linker?

# End of chapter 3